Table des matières

Labo 07 OLTP — Sous-requêtes, corrélation et ensembles sur le transactionnel 1
Le terrain, en une minute 1
Partie 1 — Les trois formes, et la corrélation 2
Partie 2 — Les questions que le GROUP BY ne sait pas poser 4
Partie 3 — Ce que le contraste vous a montré 9
Et ensuite ? 10

Labo 07 OLTP — Sous-requêtes, corrélation et ensembles sur le transactionnel

Laboratoire du Cours 07, voie OLTP · PostgreSQL + AdventureWorks OLTP · Formation SQL avancé

On refait le laboratoire OLAP, question pour question, mais sur l'autre terrain : la base transactionnelle `Adventureworks` dans PostgreSQL. Connectez-vous avec pgAdmin, DBeaver ou `psql`. Comptez une heure.

Les deux voies — Ce labo est le jumeau de `Labo_07_OLAP_SQL.md`. C'est le module où le contraste est le plus instructif : le schéma normalisé multiplie les colonnes nullables, donc le piège de `NOT IN` n'y est plus une démonstration de laboratoire, il y est un risque de tous les jours. Gardez les deux fichiers ouverts côte à côte.

Le terrain, en une minute

Le schéma normalisé éclate en tables ce que l'entrepôt rassemblait. Rappel des noms, tout en minuscules dans ce portage.

Table	Ce qu'elle contient	Colonnes utiles ici
`sales.salesorderheader`	une ligne par commande	`salesorderid`, `customerid`, `orderdate`, `totaldue`, `salespersonid`
`sales.salesorderdetail`	une ligne par ligne de commande	`salesorderid`, `productid`, `orderqty`, `unitprice`, `linetotal`
`production.product`	le catalogue	`productid`, `name`, `listprice`, `productsubcategoryid`
`production.productsubcategory`	les sous-catégories	`productsubcategoryid`, `name`, `productcategoryid`

© 2026 iAlexMG. Tous droits réservés.

Table	Ce qu'elle contient	Colonnes utiles ici
`production.productcategory`	les catégories	`productcategoryid`, `name`
`sales.customer`	les clients	`customerid`, `personid`, `storeid`

Deux colonnes nullables vont nous servir, et elles ne sont pas là par hasard : dans un schéma normalisé, une colonne nullable est la façon normale de dire « cette information n'existe pas toujours ».

Colonne	Ce que `NULL` y signifie
`production.product.productsubcategoryid`	le produit n'est rattaché à aucune sous-catégorie
`sales.salesorderheader.salespersonid`	la commande a été passée en ligne, sans vendeur

Partie 1 — Les trois formes, et la corrélation

1.1 La sous-requête scalaire

Théorie : `Theorie_07_SQL.md`, section 2.

```sql
SELECT p.name, p.listprice
FROM   production.product AS p
WHERE  p.listprice > (SELECT AVG(listprice) FROM production.product)
ORDER  BY p.listprice DESC;
```

Attendu : une centaine de lignes. Puis la version avec l'écart affiché, dans les deux écritures :

```sql
-- Version sous-requête.
SELECT p.name, p.listprice,
       p.listprice - (SELECT AVG(listprice) FROM production.product) AS ecart
FROM   production.product AS p;

-- Version fenêtre du Cours 04.
SELECT p.name, p.listprice,
       p.listprice - AVG(p.listprice) OVER () AS ecart
FROM   production.product AS p;
```

Faites ici la vérification que le labo jumeau a faite sur l'entrepôt : préfixez chaque requête de `EXPLAIN ANALYZE` et lisez le nombre de parcours de `production.product`. Le Cours 10 en fera un module entier ; pour l'instant, contentez-vous de constater la différence.

© 2026 iAlexMG. Tous droits réservés.

Un détail que l'entrepôt masquait : `listprice` vaut zéro pour certains produits qui ne sont pas vendus tels quels. La moyenne en est tirée vers le bas. Un `WHERE listprice > 0` change le résultat, et c'est le genre de décision qu'une requête doit assumer explicitement.

1.2 La sous-requête corrélée

Théorie : `Theorie_07_SQL.md`, section 3.

```
SELECT p.name,
       (SELECT COUNT(*)
        FROM   sales.salesorderdetail AS d
        WHERE  d.productid = p.productid) AS nb_lignes
FROM   production.product AS p
ORDER  BY nb_lignes DESC
LIMIT  10;
```

Attendu : une dizaine de lignes. Passez maintenant `EXPLAIN` devant, et lisez le plan.

```
Seq Scan on product p  (cost=0.00..1603554.04 rows=504 width=27)
  SubPlan 1
    ->  Aggregate  (cost=3181.62..3181.63 rows=1 width=8)
          ->  Seq Scan on salesorderdetail d  (cost=0.00..3180.46 rows=461 width=0)
                Filter: (productid = p.productid)
```

`SubPlan 1` est la boucle : la sous-requête est relancée pour chacun des 504 produits, et chaque passage balaie les lignes de commande. Le coût estimé, 1 603 554, le dit assez. PostgreSQL n'a pas décorrélé, et c'est normal : il ne le fait pas pour une sous-requête scalaire placée dans le `SELECT`.

Comparez avec un `EXISTS` corrélé, celui-là dans le `WHERE` :

```
EXPLAIN SELECT p.name
FROM   production.product AS p
WHERE  EXISTS (SELECT 1 FROM sales.salesorderdetail AS d WHERE d.productid = p.productid);
```

```
Hash Join  (cost=3186.38..3205.67 rows=263 width=19)
  Hash Cond: (p.productid = d.productid)
  ->  Seq Scan on product p
  ->  Hash  ->  HashAggregate  Group Key: d.productid
```

Aucune boucle : le moteur a reconnu le motif et l'a transformé en semi-jointure. C'est là que la décorrélation opère, et le coût tombe de trois ordres de grandeur.

Reste à réécrire la première requête vous-même, puisque le moteur ne le fera pas :

```
EXPLAIN SELECT p.name, COALESCE(d.nb, 0) AS nb_lignes
FROM   production.product AS p
```


© 2026 iAlexMG. Tous droits réservés.

```sql
  LEFT JOIN (SELECT productid, COUNT(*) AS nb
             FROM sales.salesorderdetail GROUP BY productid) AS d ON d.productid = p.productid
  ORDER  BY nb_lignes DESC
  LIMIT  10;
```

`Hash Left Join`, coût estimé 3 509 contre 1 603 554 : la même réponse, quatre cent cinquante fois moins cher. La leçon n'est donc pas « le corrélé est lent », ni « le moteur s'en occupe », mais celle-ci : lisez le plan. Il dit lequel des deux cas vous avez écrit.

1.3 La sous-requête de table

Théorie : `Theorie_07_SQL.md`, section 2.

```sql
  SELECT s.productid, s.qte_totale
  FROM  (SELECT productid, SUM(orderqty) AS qte_totale
         FROM   sales.salesorderdetail
         GROUP  BY productid) AS s
  WHERE s.qte_totale > 1000
  ORDER BY s.qte_totale DESC;
```

Réécrivez en CTE et gardez la version la plus lisible. Rappel de la règle du Cours 05 : au delà de quelques lignes, ou dès qu'elle sert deux fois, la sous-requête doit devenir une CTE.

Partie 2 — Les questions que le `GROUP BY` ne sait pas poser

2.1 Les produits jamais vendus, par anti-jointure

Théorie : `Theorie_07_SQL.md`, section 4.

```sql
  SELECT p.name, p.listprice
  FROM   production.product AS p
  WHERE  NOT EXISTS (SELECT 1
                     FROM   sales.salesorderdetail AS d
                     WHERE  d.productid = p.productid)
  ORDER  BY p.name;
```

Attendu : deux cent trente-huit lignes environ. Ce sont surtout des composants et des matières premières, qui entrent dans la fabrication sans jamais être vendus au détail.

Les deux autres écritures :

© 2026 iAlexMG. Tous droits réservés.

```sql
-- Avec NOT IN.
SELECT COUNT(*)
FROM   production.product AS p
WHERE  p.productid NOT IN (SELECT d.productid FROM sales.salesorderdetail AS d);

-- Avec LEFT JOIN et test d'absence.
SELECT COUNT(*)
FROM   production.product AS p
LEFT   JOIN sales.salesorderdetail AS d ON d.productid = p.productid
WHERE  d.productid IS NULL;
```

Les trois comptes coïncident, parce que `salesorderdetail.productid` est une clé étrangère non nullable.

L'entrepôt a une quatrième écriture, `ANTI JOIN`. Essayez-la ici, sans alias :

```sql
SELECT COUNT(*)
FROM   production.product
ANTI   JOIN sales.salesorderdetail USING (productid);
```

Attendu : 121 317, et aucune erreur. Ce n'est pas le nombre de produits jamais vendus, c'est le nombre de lignes de commande. PostgreSQL ne connaît pas `ANTI JOIN` : il a lu `ANTI` comme l'alias de `production.product`, puis il a fait une jointure interne. La requête rend exactement ce qu'on voulait écarter. Ajoutez un alias, `FROM production.product AS p ANTI JOIN … ON …`, et cette fois PostgreSQL refuse, par une erreur de syntaxe près de « ANTI ». C'est la faute la plus coûteuse du cours, parce que sous sa première forme elle ne lève rien.

Passons au cas où la colonne testée est nullable.

2.2 Le piège `NOT IN`, sur une vraie colonne nullable

Théorie : `Theorie_07_SQL.md`, section 5.

C'est ici que la voie OLTP dit quelque chose que la voie OLAP ne pouvait pas dire. Nous avons une colonne nullable authentique, `salesorderheader.salespersonid`, et une question qui a du sens : quels employés n'ont jamais pris une commande ?

La colonne est nullable pour une raison métier, et il faut la voir avant de continuer :

```sql
-- Combien de commandes n'ont pas de vendeur ?
SELECT COUNT(*) FILTER (WHERE salespersonid IS NULL) AS sans_vendeur,
       COUNT(*)                                      AS total
FROM   sales.salesorderheader;
```

Attendu : 27 659 commandes sur 31 465. Ce sont les ventes en ligne, celles-là mêmes que l'entrepôt range dans `FactInternetSales` : un client sur Internet n'a pas de vendeur, et la colonne est donc vide pour la grande majorité des lignes. Maintenant la question :

© 2026 iAlexMG. Tous droits réservés.

```sql
-- Écriture fautive.
SELECT COUNT(*) AS avec_not_in
FROM   humanresources.employee AS e
WHERE  e.businessentityid NOT IN (SELECT h.salespersonid
                                  FROM   sales.salesorderheader AS h);

-- Écriture correcte.
SELECT COUNT(*) AS avec_not_exists
FROM   humanresources.employee AS e
WHERE  NOT EXISTS (SELECT 1
                   FROM   sales.salesorderheader AS h
                   WHERE  h.salespersonid = e.businessentityid);
```

Attendu : la première renvoie **0**, la seconde renvoie **273**. Regardez bien cet écart, parce que c'est le plus coûteux de la formation. L'entreprise compte 290 employés, dont 17 prennent des commandes : 273 n'en ont jamais pris, et c'est la réponse. La première requête en annonce zéro, sans erreur, sans avertissement, et zéro est une réponse parfaitement plausible à la question posée. C'est exactement ainsi qu'un rapport faux se livre sans que personne ne s'en aperçoive.

Le mécanisme, pour mémoire : `x NOT IN (liste)` se réécrit `x <> v1 AND x <> v2 AND …`, et la liste contient ici 27 659 `NULL`. Une seule comparaison `x <> NULL` vaut inconnu, une conjonction dont un membre est inconnu ne peut jamais valoir vrai, donc aucune ligne ne passe, quelles que soient les données.

Réparez la première en ajoutant `WHERE h.salespersonid IS NOT NULL` à la sous-requête, et constatez qu'elle rend bien 273. Vérifiez au passage que la troisième écriture, `LEFT JOIN` puis `WHERE h.salespersonid IS NULL`, rend elle aussi 273 sans rien demander.

Puis rappelez-vous ce que la réparation a exigé : savoir que la colonne était nullable. `NOT EXISTS` et le `LEFT JOIN` n'exigent rien du tout et rendent le bon compte d'emblée. C'est pourquoi la règle est `NOT EXISTS`, toujours.

2.3 Les clients d'une catégorie mais pas d'une autre

Théorie : `Theorie_07_SQL.md`, section 6.

```sql
WITH ventes_par_categorie AS (
    SELECT h.customerid, c.name AS categorie
    FROM   sales.salesorderheader        AS h
    JOIN   sales.salesorderdetail        AS d  ON d.salesorderid = h.salesorderid
    JOIN   production.product            AS p  ON p.productid = d.productid
    JOIN   production.productsubcategory AS sc ON sc.productsubcategoryid = p.productsubcategoryid
    JOIN   production.productcategory    AS c  ON c.productcategoryid = sc.productcategoryid
)
SELECT customerid FROM ventes_par_categorie WHERE categorie = 'Bikes'
EXCEPT
SELECT customerid FROM ventes_par_categorie WHERE categorie = 'Accessories';
```


© 2026 iAlexMG. Tous droits réservés.

Remarquez la différence avec la voie OLAP : cinq tables au lieu de quatre, et une jointure de plus pour remonter de la ligne de commande à la commande. C'est le coût permanent du schéma normalisé, celui que l'étoile de l'entrepôt fait disparaître.

Testez ensuite `INTERSECT`, puis comparez `UNION` et `UNION ALL` sur le nombre de lignes. Et essayez `EXCEPT ALL`, qui existe ici comme sur l'entrepôt : il conserve les doublons, en soustrayant occurrence par occurrence. C'est une écriture qu'on croit souvent réservée à PostgreSQL ; elle passe des deux côtés, mais elle manque encore à plusieurs moteurs répandus.

2.4 Le top 3 par sous-catégorie, en deux écritures

Théorie : `Theorie_07_SQL.md`, sections 7 et 8.

PostgreSQL n'a pas `QUALIFY`. Il ne reste donc que deux formes, et c'est l'écart le plus visible du module.

```sql
-- Avec LATERAL. Le mot-clé est obligatoire ici, et ON true aussi.
SELECT sc.name AS sous_categorie, top.name, top.listprice
FROM   production.productsubcategory AS sc
LEFT   JOIN LATERAL (SELECT p.name, p.listprice
                     FROM   production.product AS p
                     WHERE  p.productsubcategoryid = sc.productsubcategoryid
                     ORDER  BY p.listprice DESC
                     LIMIT  3) AS top ON true
ORDER  BY sous_categorie, top.listprice DESC;

-- Avec une fenêtre et une CTE, la forme portable.
WITH classe AS (
    SELECT sc.name AS sous_categorie, p.name, p.listprice,
           ROW_NUMBER() OVER (PARTITION BY sc.productsubcategoryid
                              ORDER BY p.listprice DESC) AS rang
    FROM   production.product AS p
    JOIN   production.productsubcategory AS sc
           ON sc.productsubcategoryid = p.productsubcategoryid
)
SELECT sous_categorie, name, listprice
FROM   classe
WHERE  rang <= 3
ORDER  BY sous_categorie, listprice DESC;
```

Faites l'expérience qui compte : remplacez `LEFT JOIN LATERAL … ON true` par une simple virgule et `LATERAL`, et comparez le nombre de lignes. Les sous-catégories vides disparaissent. Le `LEFT` n'est pas décoratif.

2.5 Une anti-jointure de contrôle qualité

Théorie : `Theorie_07_SQL.md`, section 4.

© 2026 iAlexMG. Tous droits réservés.

```sql
-- Des lignes de commande dont le produit n'existe pas au catalogue ?
SELECT COUNT(*) AS lignes_orphelines
FROM   sales.salesorderdetail AS d
WHERE  NOT EXISTS (SELECT 1 FROM production.product AS p WHERE p.productid = d.productid);

-- Des commandes dont le client n'existe pas ?
SELECT COUNT(*) AS commandes_orphelines
FROM   sales.salesorderheader AS h
WHERE  NOT EXISTS (SELECT 1 FROM sales.customer AS c WHERE c.customerid = h.customerid);
```

Attendu : zéro dans les deux cas, et pour une raison que l'entrepôt n'a pas. Ici, des **clés étrangères** l'imposent au niveau de la base : le moteur refuserait une ligne orpheline. Dans l'entrepôt de la formation, rien ne l'interdit : le script du Cours 00 n'y déclare aucune clé, et DuckDB n'a donc rien à vérifier, lui qui refuse une ligne orpheline dès que la clé étrangère est déclarée (Cours 03). Sur Snowflake, d'après sa documentation, une clé étrangère déclarée sur une table ordinaire ne serait même pas vérifiée.

C'est le contraste le plus important de ce laboratoire. La même requête de contrôle est une formalité côté transactionnel et une nécessité côté entrepôt.

Contrôle de cohérence — Gardez cette requête. C'est le geste qu'un ingénieur analytique répète partout : « ai-je des transactions qui référencent un instrument inconnu ? », « des ventes dont le magasin n'existe plus dans le référentiel ? ». La forme ne change jamais, seules les tables changent. Et souvenez-vous que dans l'entrepôt, personne ne vous protège.

2.6 Rapprocher deux ensembles, dans les deux sens

Théorie : `Theorie_07_SQL.md`, section 6.

Les clients de 2024 et de 2025, les deux dernières années du portage :

```sql
WITH clients AS (
    SELECT DISTINCT customerid, EXTRACT(YEAR FROM orderdate) AS annee
    FROM   sales.salesorderheader
)
SELECT (SELECT COUNT(*) FROM clients WHERE annee = 2024) AS en_2024,
       (SELECT COUNT(*) FROM clients WHERE annee = 2025) AS en_2025,
       (SELECT COUNT(*) FROM (SELECT customerid FROM clients WHERE annee = 2024
                              EXCEPT
                              SELECT customerid FROM clients WHERE annee = 2025) AS x) AS perdus,
       (SELECT COUNT(*) FROM (SELECT customerid FROM clients WHERE annee = 2025
                              EXCEPT
                              SELECT customerid FROM clients WHERE annee = 2024) AS x) AS gagnes;
```

Attendu : 11 145 clients en 2024, 10 303 en 2025, 8 363 perdus et 7 521 gagnés. Notez l'alias `x` après chaque sous-requête : PostgreSQL 18 s'en passe, mesuré, mais le nommer reste la forme qui passe sur tous les moteurs. Le portage ne raconte pas la même histoire que l'entrepôt : ses années sont décalées, et il compte les ventes aux revendeurs.

© 2026 iAlexMG. Tous droits réservés.

Puis la copie fautive, cette fois sur une centaine de commandes dont une ligne sur dix est chargée deux fois :

```
WITH source AS (
    SELECT salesorderid, salesorderdetailid, linetotal
    FROM   sales.salesorderdetail
    WHERE  salesorderid BETWEEN 50000 AND 50100
),
cible AS (
    SELECT * FROM source
    UNION ALL
    SELECT * FROM source WHERE salesorderdetailid % 10 = 0
)
SELECT (SELECT COUNT(*) FROM source) AS lignes_source,
       (SELECT COUNT(*) FROM cible)  AS lignes_cible,
       (SELECT COUNT(*) FROM (SELECT * FROM cible EXCEPT     SELECT * FROM source) AS x) AS en_trop_except,
       (SELECT COUNT(*) FROM (SELECT * FROM cible EXCEPT ALL SELECT * FROM source) AS x) AS en_trop_except_all;
```

Attendu : 101 lignes à la source, 111 à la copie, 0 par `EXCEPT`, 10 par `EXCEPT ALL`. Le même verdict que sur l'entrepôt : seul `EXCEPT ALL` voit les lignes en trop.

Partie 3 — Ce que le contraste vous a montré

Théorie : `Theorie_07_SQL.md`, section 8.

Sujet	**PostgreSQL, ce labo**	**L'entrepôt, `Labo_07_OLAP_SQL.md`**
Top N par groupe	`LATERAL` ou fenêtre plus CTE. Pas de raccourci	`QUALIFY` fait tenir la chose en une seule requête
`LATERAL`	`LEFT JOIN LATERAL (…) ON true`, mot-clé et `ON` obligatoires	le mot-clé est facultatif ; la virgule suffit, mais perd les lignes de la table de gauche sans correspondance
Le piège `NOT IN`	se démontre sur une **vraie** colonne nullable du schéma, `productsubcategoryid`	se démontre sur une liste littérale, faute de nullable évident
`EXCEPT`	`EXCEPT` et `EXCEPT ALL`	les deux aussi ; `MINUS`, lui, n'existe ni ici ni là
`ANTI JOIN`	inconnu : sans alias, `ANTI` devient l'alias de la table, et la requête rend la jointure interne sans erreur	l'anti-jointure en toutes lettres, avec `SEMI JOIN`

© 2026 iAlexMG. Tous droits réservés.

Sujet	PostgreSQL, ce labo	L'entrepôt, `Labo_07_OLAP_SQL.md`
Les lignes orphelines	impossibles, les clés étrangères les interdisent	possibles, rien ne les empêche. Le contrôle devient obligatoire
Le chemin jusqu'à la catégorie	cinq tables jointes	quatre, l'étoile a déjà rapproché le contexte

La leçon à retenir de ce contraste dépasse la syntaxe. Le transactionnel vous protège par des contraintes et vous complique la vie par sa normalisation ; l'entrepôt vous simplifie les jointures et vous laisse seul responsable de la cohérence. L'anti-jointure de contrôle est l'outil qui compense la seconde moitié de ce marché.

Et ensuite ?

Vous avez maintenant vu les deux terrains sur les mêmes questions. Le `LATERAL` que vous venez d'écrire est le geste exact de l'as-of join du Cours 09, où vous demanderez « le dernier prix connu avant cette date » au lieu de « les trois plus chers de ce groupe ».

Prochaine étape : les exercices, dans `Exercices_corriges_07_SQL.md`. Plusieurs énoncés supposent que vous avez tapé les requêtes des deux laboratoires.

© 2026 iAlexMG. Tous droits réservés.